

Repository Update (CountryRepository.java)

```
package com.cognizant.ormlearn.repository;

import java.util.List;
import org.springframework.data.jpa.repository.JpaRepository;
import org.springframework.stereotype.Repository;
import com.cognizant.ormlearn.model.Country;

@Repository
public interface CountryRepository extends JpaRepository<Country, String> {

    List<Country> findByNameContaining(String text);
    List<Country> findByNameContainingOrderByNameAsc(String text);
    List<Country> findByNameStartingWith(String prefix);
}
```

Service Layer Update (CountryService.java)

```
package com.cognizant.ormlearn.service;

import java.util.List;

import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.stereotype.Service;
import org.springframework.transaction.annotation.Transactional;
import com.cognizant.ormlearn.model.Country;
import com.cognizant.ormlearn.repository.CountryRepository;

@Service
public class CountryService {

    @Autowired
    private CountryRepository countryRepository;
```

```
@Transactional(readOnly = true)

public List<Country> getAllCountries() {

    return countryRepository.findAll();

}
```

```
@Transactional(readOnly = true)

public List<Country> searchCountriesByText(String text) {

    return countryRepository.findByNameContaining(text);

}
```

```
@Transactional(readOnly = true)

public List<Country> searchCountriesByTextSorted(String text) {

    return countryRepository.findByNameContainingOrderByNameAsc(text);

}
```

```
@Transactional(readOnly = true)

public List<Country> getCountriesStartingWith(String prefix) {

    return countryRepository.findByNameStartingWith(prefix);

}

}
```

Persistence Model Class (Stock.java)

```
package com.cognizant.ormlearn.model;
```

```
import java.math.BigDecimal;
```

```
import java.util.Date;

import javax.persistence.Column;

import javax.persistence.Entity;

import javax.persistence.GeneratedValue;

import javax.persistence.GenerationType;

import javax.persistence.Id;

import javax.persistence.Table;

import javax.persistence.Temporal;

import javax.persistence.TemporalType;
```

```
@Entity
```

```
@Table(name = "stock")
```

```
public class Stock {
```

```
    @Id
```

```
    @GeneratedValue(strategy = GenerationType.IDENTITY)
```

```
    @Column(name = "st_id")
```

```
    private int id;
```

```
    @Column(name = "st_code")
```

```
    private String code;
```

```
    @Column(name = "st_date")
```

```
    @Temporal(TemporalType.DATE)
```

```
    private Date date;
```

```
@Column(name = "st_open")
```

```
private BigDecimal open;
```

```
@Column(name = "st_close")
```

```
private BigDecimal close;
```

```
@Column(name = "st_volume")
```

```
private Long volume;
```

```
public Stock() {}
```

```
public int getId() { return id; }
```

```
public void setId(int id) { this.id = id; }
```

```
public String getCode() { return code; }
```

```
public void setCode(String code) { this.code = code; }
```

```
public Date getDate() { return date; }
```

```
public void setDate(Date date) { this.date = date; }
```

```
public BigDecimal getOpen() { return open; }
```

```
public void setOpen(BigDecimal open) { this.open = open; }
```

```
public BigDecimal getClose() { return close; }
```

```

    public void setClose(BigDecimal close) { this.close = close; }

    public Long getVolume() { return volume; }

    public void setVolume(Long volume) { this.volume = volume; }

    @Override

    public String toString() {

        return String.format("| %-7s | %s | %7s | %8s | %10d |", code, date, open,
close, volume);

    }

}

```

Data Repository Component (StockRepository.java)

```

package com.cognizant.ormlearn.repository;

import java.math.BigDecimal;

import java.util.Date;

import java.util.List;

import org.springframework.data.jpa.repository.JpaRepository;

import org.springframework.stereotype.Repository;

import com.cognizant.ormlearn.model.Stock;

@Repository

public interface StockRepository extends JpaRepository<Stock, Integer> {

    List<Stock> findByCodeAndDateBetween(String code, Date startDate, Date
endDate);

    List<Stock> findByCodeAndCloseGreaterThan(String code, BigDecimal price);

```

```
List<Stock> findTop3ByOrderByVolumeDesc();  
  
List<Stock> findTop3ByCodeOrderByCloseAsc(String code);  
  
}
```

Execution Runner File (OrmLearnApplication.java)

```
package com.cognizant.ormlearn;  
  
  
import java.math.BigDecimal;  
  
import java.text.SimpleDateFormat;  
  
import java.util.Date;  
  
import java.util.List;  
  
import org.slf4j.Logger;  
  
import org.slf4j.LoggerFactory;  
  
import org.springframework.boot.SpringApplication;  
  
import org.springframework.boot.autoconfigure.SpringBootApplication;  
  
import org.springframework.context.ApplicationContext;  
  
import com.cognizant.ormlearn.model.Country;  
  
import com.cognizant.ormlearn.model.Stock;  
  
import com.cognizant.ormlearn.service.CountryService;  
  
import com.cognizant.ormlearn.repository.StockRepository;  
  
  
@SpringBootApplication  
  
public class OrmLearnApplication {
```

```
private static final Logger LOGGER =
LoggerFactory.getLogger(OrmLearnApplication.class);

private static CountryService countryService;

private static StockRepository stockRepository;


public static void main(String[] args) {

    ApplicationContext context =
SpringApplication.run(OrmLearnApplication.class, args);

    LOGGER.info("Inside main");


    countryService = context.getBean(CountryService.class);
    stockRepository = context.getBean(StockRepository.class);


    testCountryQueries();

    testStockQueries();

}


private static void testCountryQueries() {

    LOGGER.info("--- Testing Country Query Methods ---");


    LOGGER.debug("Countries containing 'ou':");

    List<Country> c1 = countryService.searchCountriesByText("ou");

    c1.forEach(c -> LOGGER.debug(c.toString()));


    LOGGER.debug("Countries containing 'ou' (Sorted Alphabetically):");

    List<Country> c2 = countryService.searchCountriesByTextSorted("ou");
```

```

c2.forEach(c -> LOGGER.debug(c.toString()));

LOGGER.debug("Countries starting with 'Z':");

List<Country> c3 = countryService.getCountriesStartingWith("Z");

c3.forEach(c -> LOGGER.debug(c.toString()));
}

private static void testStockQueries() {

    LOGGER.info("--- Testing Stock Query Methods ---");

    try {

        SimpleDateFormat df = new SimpleDateFormat("yyyy-MM-dd");

        LOGGER.debug("FB Stocks - Sept 2019:");

        Date start = df.parse("2019-09-01");

        Date end = df.parse("2019-09-30");

        List<Stock> fbStocks = stockRepository.findByCodeAndDateBetween("FB",
start, end);

        fbStocks.forEach(s -> LOGGER.debug(s.toString()));

        LOGGER.debug("Google Stocks > 1250:");

        List<Stock> googlStocks =
stockRepository.findByCodeAndCloseGreaterThan("GOOGL", new
BigDecimal("1250.00"));

        googlStocks.forEach(s -> LOGGER.debug(s.toString()));

        LOGGER.debug("Top 3 Highest Volume Transactions:");

```



```
List<Stock> topVolume =
stockRepository.findTop3ByOrderByVolumeDesc();

topVolume.forEach(s -> LOGGER.debug(s.toString()));

LOGGER.debug("Top 3 Lowest Netflix Stocks:");

List<Stock> lowestNflx =
stockRepository.findTop3ByCodeOrderByCloseAsc("NFLX");

lowestNflx.forEach(s -> LOGGER.debug(s.toString()));

} catch (Exception e) {

    LOGGER.error("Error executing stock queries", e);

}

}

}
```